

A Small, Calibrated Verifier That Learns From Outcomes: Judging Coding-Agent Runs Locally with Gemma 4, and the Split Leakage That Inflates Verifier Results

John Soliva

D1A project, <https://github.com/jonpol01/d1a>

DRAFT — October 5, 2026

Abstract

A coding agent cannot see the hidden tests that decide whether its patch resolves an issue, so it must guess when it is done. We study a small verifier that runs locally and gives that guess a calibrated probability: D1A-E4B, Gemma 4 E4B with a LoRA adapter and a pointer head. It reads the issue, the agent’s final patch and the end of its run, and answers “does this patch resolve the issue?” in one forward pass (median 1.5s on a laptop). On 921 public SWE-agent runs from 83 repositories it never saw, the trained verifier reaches AUROC 0.807 (calibration error 0.039), and adding it to cheap trajectory heuristics raises AUROC from 0.768 to 0.819 (+0.051, 95% CI [+0.002, +0.101], bootstrap over repositories). Two findings shape how such verifiers should be built and evaluated. First, *split leakage*: runs of the same issue share most of their outcome, so a verifier evaluated with runs of one issue on both sides of the split can score well by recognising the issue rather than judging the run. In a first shard of 6,670 runs, a text verifier reaches AUROC 0.96 under a random split of runs but 0.70 on unseen repositories, and ranks runs *within* an issue no better than chance [pending: full 80k-run numbers]. Second, *learning from outcomes needs a gate*: in a loop that recalibrates and retrains on outcomes as they arrive, recalibration alone halved the calibration error on unseen repositories, while the first retrained candidate was worse and was rejected by a statistical promotion gate [pending: rounds 2–3]. All data is public, and all code is open.

1 Introduction

LLM coding agents edit repositories in long runs of tool calls and then submit a patch. At submission time the agent does not know whether the hidden tests will pass. A verifier that estimates $P(\text{resolved})$ supports three decisions: whether to submit or keep working, which of several runs to submit, and which runs to keep as training data. Each decision needs the probability to mean what it says, not only to rank runs, and in many settings the verifier must be cheap and local.

This is the setting of the Gemma 4 Developer Agent challenge, where an agent runs offline on a fixed Gemma 4 model and is scored by hidden tests it never sees, under a time budget per task: a small verifier on the same hardware is what such an agent can afford to ask. Verifiers for coding agents already exist (Section 2); we do not claim a new kind of verifier. Our contributions are narrower:

1. **A small, local, calibrated verifier, evaluated on unseen repositories.** We report calibration (ECE, reliability, risk–coverage) next to ranking, all held out by repository, for a 4B-class model that answers in one forward pass on a laptop (Section 3).

2. **A measurement of split leakage in verifier evaluation.** On the same runs, we compare random-run, by-issue and by-repository splits, and report the AUROC inflation and the within-issue AUROC that best-of- k selection actually depends on (Section 4).
3. **An outcome feedback loop with a statistical promotion gate,** released as a library (`d1a.feedback`), and its first measured rounds (Section 5).

2 Related Work

Verifiers for software-engineering agents. SWE-Gym trains verifiers on agent trajectories and uses them for inference-time scaling, selecting among sampled trajectories [1]. SWE-RM is a 30B execution-free reward model that treats classification accuracy and calibration as crucial for reinforcement learning [2]. Agentic Rubrics derive contextual rubrics as a verification signal [3]. Compared with these, our verifier is small enough to run on a laptop and is evaluated for calibration on repositories it never saw.

Early failure prediction. FailFast-RestartSmart trains a 0.6B monitor on trajectory prefixes and restarts runs predicted to fail [4]; EarlyEval predicts outcomes early with gradient-boosted classifiers to stop runs and save evaluation cost [5]. Our anytime experiment is a weaker instance of the same idea and is reported as a contrast.

Self-improvement with gates. Audited skill-graph self-improvement promotes a candidate only after verifier-backed replay [6], and self-authored verification is shown to be unreliable, motivating an external acceptance gate [7]. Verification also has to keep up with the policy it judges [8]. Our promotion gate is external in the same sense but is driven by real outcomes and decides with a bootstrap interval over repositories.

3 The Verifier

Model. D1A [11] is a decision model built on Kev [10]: a causal LM backbone with a LoRA adapter runs one prefill over a document and typed questions under a block-causal mask, and a pointer head scores each option against the question’s decision token. The backbone here is Gemma 4 E4B. D1A-E4B v0.4, released before this work, was trained on general decisions and pull-request labelling, never on agent runs.

Input and question. The document holds the issue (up to 3,000 characters), the agent’s final patch (up to 6,000) and the last steps of its run (up to 3,000; each step up to 800). The question is a yes/no: “does the patch correctly fix the issue, so that the repository’s tests for it would pass?”. Inputs are 1.5k tokens at the median and 5.5k at most.

Data. We use nebius/SWE-agent-trajectories (CC-BY-4.0): 80,036 SWE-agent runs on 3,591 issues, each with **target** (resolved by the hidden tests), the trajectory and the patch. The test logs are never read. Repositories are assigned to train (70%), dev (10%) and test (20%) by a hash of the name, so no repository is in two splits. Training uses 2,000 runs from 377 repositories (at most four per issue); dev holds 1,142 runs (8 per issue) from 56 repositories; test holds 921 runs (8 per issue) from 83 repositories, 12.4% resolved.

Training and calibration. One LoRA run continues D1A-E4B v0.4 for 100 optimizer steps of 8 runs (800 runs, learning rate $5 \cdot 10^{-5}$), 2.5 h on an Apple M1 Max at no cost. Calibration (Platt scaling on the logit) is fitted on dev only. The heuristics baseline is a logistic regression over six trajectory features (patch size, a clean “submitted” exit, number of steps, error observations, printed success, test-file edits), fitted on training-split runs only.

Results on unseen repositories. Table 1. Zero-shot, D1A ranks reasonably but is badly overconfident; after training it ranks better than the heuristics and is nearly calibrated before any recalibration.

	AUROC	ECE	Brier
Heuristics (logistic regression)	0.768	0.019	0.100
D1A-E4B v0.4 zero-shot, raw	0.742	0.632	0.514
recalibrated on dev	0.742	0.043	0.103
+ heuristics	0.765	0.047	0.101
D1A-E4B + verifier LoRA (r1), raw	0.807	0.058	0.098
recalibrated on dev	0.807	0.039	0.097
+ heuristics	0.819	0.037	0.094

Table 1: Verifier quality on 921 runs from 83 repositories unseen in training (12.4% resolved; base-rate Brier 0.108). AUROC differences against the heuristics, 95% CI from a bootstrap over repositories: r1 +0.039 [−0.030, +0.109]; heuristics + r1 +0.051 [+0.002, +0.101]; heuristics + zero-shot −0.003 [−0.043, +0.040].

A gain that did not replicate. Under 5-fold cross-validation grouped by repository on a separate random sample of 600 runs, zero-shot D1A appeared to add +0.069 AUROC to the heuristics (95% CI [+0.020, +0.114]). On the fixed test split it added nothing (−0.003). Both evaluations were grouped by repository, so this is not leakage but a gain that did not survive a second sample; we report it as a caution.

What the probability is good for.

- *Training-data filtering.* Keeping the 10% of test runs that the verifier trusts most gives 40.2% truly resolved runs (37 runs), against 22.8% for the heuristics (21), 19.6% for the agent’s own clean “submitted” exit and 15.2% at random. At 20% and 30% yield, heuristics + r1 keeps 32.6% and 31.2% resolved runs against 29.9% and 27.5% for the heuristics.
- *Best-of- k .* Picking one of 8 recorded runs per issue (97 issues), heuristics + r1 submits a resolving run 19.6% of the time, against 13.4% for the heuristics, 14.7% at random and 26.8% for an oracle: +0.062 [0.000, +0.124] over the heuristics (bootstrap over issues). At $k = 2$ and $k = 4$ the gains (+0.009, +0.018) are not significant.
- *Latency.* Median 1.48 s and p95 2.79 s per run (MLX, 8-bit, M1 Max).

Anytime verification is weak here. Scoring an unfinished run at its tenth action (patch absent, final exit hidden), trajectory heuristics are at chance (AUROC 0.40), while zero-shot D1A carries signal (0.66; +0.26 [+0.08, +0.44] over the heuristics). Replaying stop-and-restart policies on recorded runs under an action budget, the verifier gains at most 1.3 points (12.8% vs 11.5% for never stopping at a 90-action budget; oracle 15.9%). FailFast-RestartSmart reports much larger

gains with a dedicated 0.6B prefix monitor [4]; our verifier, trained only on finished runs, is not a substitute for one.

4 Split Leakage in Verifier Evaluation

Agent runs of the same issue share most of their outcome: some issues are resolved by nearly every run, others by none. A learned verifier evaluated with runs of one issue in both training and test can therefore score well by recognising the issue. We measure this directly on the same runs under three 5-fold protocols: random runs, grouped by issue, and grouped by repository. Four verifiers are refitted per fold: an *issue prior* (the resolved rate of the same issue’s training runs; pure leakage by construction), the heuristics, a TF-IDF model of the issue and patch text, and both together. Besides global AUROC we report *within-issue AUROC*, the ranking of runs of the same issue against each other, which is what best-of- k selection uses and what an issue prior cannot improve.

Verifier	random runs	by issue	by repository
Issue prior	0.95	0.42	0.42
Heuristics	0.75	0.72	0.72
Text (TF-IDF)	0.96	0.71	0.70
Heuristics + text	0.95	0.76	0.76

Table 2: Global AUROC of out-of-fold predictions under three splits of the same runs. [pending: replace with the 80,036-run result; these are from one shard, 6,670 runs on 307 issues.] Inflation over the by-repository split (95% CI, bootstrap over repositories): issue prior +0.53 [+0.42, +0.64], text +0.26 [+0.20, +0.32], heuristics + text +0.19 [+0.15, +0.24], heuristics +0.03 [+0.01, +0.04]; by-issue splits show none.

Under a random split the text model’s global AUROC is 0.96, yet its within-issue AUROC is 0.50: it has learned which issues are easy, not which runs are right. Grouping by issue removes the inflation; grouping by repository adds little beyond that in this data. [pending: confirm on the full set and on OpenHands and SWE-smith trajectories.]

5 Learning From Outcomes

D1A’s decisions often get a ground truth later. `d1a.feedback` logs each decision with the model version that made it, records the outcome when it arrives, and improves the model in three steps, cheapest first: (1) an *outcome calibrator* refits each yes/no probability on the outcomes (Platt scaling [9], which also corrects a shifted base rate); (2) a small LoRA update trains on the resolved decisions; (3) a *promotion gate* replaces the current model only if the candidate’s held-out log loss is lower with a 95% bootstrap interval (over repositories) entirely below zero and no frozen evaluation suite regressed by more than one point. This mirrors the external acceptance gates of [6, 7], here driven by real outcomes rather than self-authored tests.

Rounds. Starting from r1, batches of 300 unseen runs (train-split repositories of other shards) arrive with their outcomes. Each round, 80% of the batch (by repository) becomes training records and 20% a calibration slice; each model is recalibrated only on slices it never trained on; the gate decides on dev repositories; the served model is measured on test repositories.

Round	outcomes	test AUROC	test ECE	gate
0 (r1, no feedback)	0	0.808	0.068	–
1	300	0.808	0.036	rejected: log loss +0.026 [+0.016, +0.058]
2	600	[pending:]	[pending:]	[pending:]
3	900	[pending:]	[pending:]	[pending:]

Table 3: The served verifier after each round on test repositories (four runs per issue). In round 1 recalibration alone halved the calibration error, while the retrained candidate was worse on dev and the gate kept the current model.

Adapting to a new agent. A verifier trained on one agent meets new agents as they appear, and a fixed reward function falls behind as the policy improves [8]. [pending: SWE-agent → OpenHands (nebius/SWE-rebench-openhands-trajectories): AUROC and ECE after $N \in \{25, 50, 100, 200, 400\}$ outcomes from the new agent, recalibration alone vs recalibration + LoRA, with gated promotion.]

6 Limitations

One agent family (SWE-agent with Llama models) and one public dataset carry the main results so far; the cross-agent study is pending. The challenge provides per-repository code graphs and embeddings; this verifier does not use them, because it judges a finished run from its issue, patch and trajectory, and in a separate study of ours a D1A localiser over the provided graphs did not beat BM25 at finding the code to change. The verifier sees truncated inputs (issue, patch, last steps). Test-split resolve rates are low (12.4%), which widens intervals on best-of- k and filtering numbers. The leakage table is preliminary. We report the zero-shot gain that did not replicate rather than only the one that did.

7 Reproducibility

All code is in <https://github.com/jonpol01/d1a> (recipes/swe-verifier: zero_shot.py, to_records.py, evaluate.py, best_of_k.py, budget_sim.py, leakage.py, self_improve.py; library d1a.feedback). Data is public (CC-BY-4.0, MIT). Training and evaluation ran on one Apple M1 Max laptop at no cost.

References

- [1] J. Pan, X. Wang, G. Neubig, N. Jaitly, H. Ji, A. Suhr, Y. Zhang. Training Software Engineering Agents and Verifiers with SWE-Gym. arXiv:2412.21139, 2024.
- [2] K. Shum, B. Hui, J. Chen, L. Zhang, X. W., J. Yang, Y. Huang, J. Lin, J. He. SWE-RM: Execution-free Feedback for Software Engineering Agents. arXiv:2512.21919, 2025. (The author “X. W.” is listed in this form on arXiv.)
- [3] M. Raghavendra, A. Gunjal, B. Liu, Y. He. Agentic Rubrics as Contextual Verifiers for SWE Agents. arXiv:2601.04171, 2026.
- [4] C. Wang, Y. Lyu, J. He, Z. Yang, C. Zhong, Y. Harel, D. Lo. Fail-Fast, Restart-Smart: Early Failure Prediction and Restart for SWE Agentic Tasks. arXiv:2608.03222, 2026.

- [5] Y. Shi, Z. Sun, J. Dong, C. Wan, D. Lo, X. Gu. EarlyEval: Cheaper Agent Evaluation via Early Outcome Prediction. arXiv:2609.02783, 2026.
- [6] K. Huang, J. Huang. Audited Skill-Graph Self-Improvement for Agentic LLMs via Verifiable Rewards, Experience Synthesis, and Continual Memory. arXiv:2512.23760, 2025.
- [7] D. Guo, C. Cao, F. Yuan, Y. Wang, Y. Wang, D. Wang. Self-Authored Verification Is Unreliable in Heuristic Self-Improving Agents. arXiv:2607.24300, 2026.
- [8] B. Wang et al. The Verification Horizon: No Silver Bullet for Coding Agent Rewards. arXiv:2606.26300, 2026.
- [9] J. C. Platt. Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods. In *Advances in Large Margin Classifiers*, MIT Press, 1999.
- [10] J. Palmer. Kev. <https://github.com/jaredpalmer/kev> (Apache-2.0).
- [11] J. Soliva. D1A. <https://github.com/jonpol01/d1a> (Apache-2.0).